

CO4-AT1 – Question 28

Name: R.Gautham Krishna

Register No: 192421128

Subject: Design and Analysis of Algorithms

Faculty: Dr. F. Mary Harin Fernandez

Q28. Weighted Interval Scheduling

1. Problem Statement

A set of intervals is given, where each interval has a start time, end time, and associated weight. The objective is to select a subset of non-overlapping intervals such that the total weight is maximized.

The problem is solved using Dynamic Programming, where the optimal solutions of smaller subproblems are stored and reused.

For example, the given intervals are:

- (1, 3, 5)
- (2, 5, 6)
- (4, 6, 5)

The intervals (1,3,5) and (4,6,5) do not overlap, so their total weight is:

$$5 + 5 = 10$$

Therefore, the maximum weight is 10.

2. Objective

The main objectives are:

- To understand the Weighted Interval Scheduling problem.
- To implement the solution using Dynamic Programming.
- To select non-overlapping intervals.
- To maximize the total weight.
- To analyze the time and space complexity of the solution.

3. Input

$n = 3$

Intervals:

(1, 3, 5)

(2, 5, 6)

(4, 6, 5)

Where each interval represents:

(start, end, weight)

4. Output

Max Weight = 10

5. Approach Used

The problem is solved using Dynamic Programming.

First, the intervals are sorted according to their end times.

For every interval, two possibilities are considered:

1. Include the current interval and add its weight to the best compatible previous interval.
2. Exclude the current interval and use the previous optimal result.

The maximum of these two choices is stored in the DP table.

6. Dynamic Programming Concept

Let:

$dp[i]$ represent the maximum weight that can be obtained using the first i intervals.

For each interval:

- Find the latest interval that finishes before the current interval starts.
- Include the current interval.
- Exclude the current interval.
- Choose the maximum of both.

The recurrence relation is:

$$dp[i] = \max(dp[i-1], \text{weight}[i] + dp[\text{previous}])$$

where previous is the last non-overlapping interval.

7. Step-by-Step Analysis

Given:

(1,3,5)

(2,5,6)

(4,6,5)

Interval 1

(1,3,5)

Maximum weight:

$$dp[1] = 5$$

Interval 2

(2,5,6)

It overlaps with interval 1.

Choose the maximum:

$$\max(5, 6) = 6$$

Therefore:

$$dp[2] = 6$$

Interval 3

(4,6,5)

It is compatible with interval 1 because interval 1 ends at 3 and interval 3 starts at 4.

Therefore:

$$5 + 5 = 10$$

Compare with excluding interval 3:

$$\max(6, 10) = 10$$

Therefore:

$$dp[3] = 10$$

The selected intervals are:

(1,3,5)

(4,6,5)

Total weight:

$5 + 5 = 10$

8. DP Table

Position Interval Maximum Weight

0	Empty	0
1	(1,3,5)	5
	(2,5,6)	6
3	(4,6,5)	10

Final result:

$dp[3] = 10$

9. Source Code

```
main.py Visualize Run
1 def weighted_interval_scheduling(intervals):
2     intervals.sort(key=lambda x: x[1])
3     n = len(intervals)
4     dp = [0] * n
5     for i in range(n):
6         start, end, weight = intervals[i]
7         j = i - 1
8         while j >= 0 and intervals[j][1] > start:
9             j -= 1
10        include = weight
11        if j >= 0:
12            include += dp[j]
13        exclude = dp[i - 1] if i > 0 else 0
14        dp[i] = max(include, exclude)
15    return dp[n - 1]
16 n = 3
17 intervals = [
18     (1, 3, 5),
19     (2, 5, 6),
20     (4, 6, 5)
21 ]
22 result = weighted_interval_scheduling(intervals)
23 print("Max Weight =", result)
```

10.

Execution / Output

Output

Max Weight = 10

11. Handling Overlapping Intervals

The algorithm checks whether intervals overlap.

For example:

(1,3,5)

(2,5,6)

These intervals overlap because the second interval starts before the first interval ends.

Therefore, they cannot both be selected.

However:

(1,3,5)

(4,6,5)

do not overlap, so both can be selected.

12. Recurrence Relation

For every interval i :

$$dp[i] = \max(\begin{aligned} & dp[i-1], \\ & weight[i] + dp[previous] \end{aligned})$$

Where:

- $dp[i-1]$ → Exclude the current interval.
- $weight[i] + dp[previous]$ → Include the current interval.
- $previous$ → Latest interval that does not overlap with the current interval.

13. Time Complexity

If the intervals are sorted and the previous compatible interval is searched linearly:

Time Complexity – $O(n^2)$

With binary search for the previous compatible interval:

Time Complexity – $O(n \log n)$

14. Space Complexity

A DP table of size n is maintained.

Space Complexity – $O(n)$

15. Advantages

- Uses an efficient Dynamic Programming approach.
- Maximizes the total weight of selected intervals.
- Avoids selecting overlapping intervals.
- Reuses previously calculated results.
- Works efficiently for a large number of intervals.
- Can be optimized using binary search.

16. Result

The Weighted Interval Scheduling problem was successfully solved using the Dynamic Programming technique.

For the input:

(1,3,5)

(2,5,6)

(4,6,5)

The optimal intervals are:

(1,3,5)

(4,6,5)

Therefore:

Max Weight = 10

17. Conclusion

The Weighted Interval Scheduling problem demonstrates the effective use of Dynamic Programming to select non-overlapping intervals while maximizing their total weight. By considering both inclusion and exclusion of each interval, the optimal solution can be obtained efficiently.

Time Complexity – $O(n^2)$ | Space Complexity – $O(n)$

Optimized version: Time Complexity – $O(n \log n)$ | Space Complexity – $O(n)$